PS2版Tetris画像ROM
不可逆ニブル復元の考え方

4ニブル単位の構造とFlash Point比較から、失われたFマーカーを再構成する仕組み

結論

概ね、4ニブル単位の構造を見ることと、Flash Pointに残る正常なF配置を比較することの組み合わせで復元しています。単純に4ファイルを重ねて差分を抜き出したわけではありません。

> **共通画像のパターン一致を手掛かりにFマーカーの配置規則を推定し、その規則をFlash Point全69画像で検証してからTetrisへ適用した、という理解が最も正確です。**

1. 核心は4ニブル単位で見ること

System 16Bのスプライトは16ビット、つまり4ニブルずつ読み取られます。

```
[ nibble 0 ] [ nibble 1 ] [ nibble 2 ] [ nibble 3 ]
```

通常方向では、4番目のニブルがFなら、その行の読み出しを終了します。そのため元ROMには概ね次のマーカー構造があります。

```
描画部分より左 : Fxxx
描画部分より右 : xxxF
```

PS2版ではFが0へ潰れているため、たとえば F00F と 000F は、どちらも 0000 になります。ニブル単体では元の0とFを区別できません。

しかし、画像を行単位に区切り、各行の最初と最後の非ゼロ画素を調べれば、描画領域の外側に置かれていたFを規則的に戻せます。

```
PS2側 : 0000 0000 0123 4500 0000
復元例 : F00F F00F 0123 450F 000F
```

2. 4ファイルはそれぞれ役割が異なる

ファイル	役割
tetris.obj	Fが0へ潰れた復元対象
tetrisd.p94	Tetrisの58画像の開始位置・幅・高さを取得
flashp.obj	Fが残っている正常な比較資料
flashpd.p94	Flash Pointの69画像の開始位置・幅・高さを取得

これらを単純に重ねるのではなく、配置情報で画像を正しく区切り、正常データから確認した規則を復元対象へ適用します。

3. Flash Pointで規則を確認する

まず、Fが残っているFlash Pointの画像から、Fだけを意図的に0へ変換します。その模擬的な欠損データに復元規則を適用し、元のFlash Point画像とバイト単位で一致するかを確認します。

```
flashp.obj
↓ Fを0へ変換
模擬欠損データ
↓ 4ニブル単位の規則でFを補完
元のflashp.objと比較
```

この検証から、次の規則が確認されています。

・左側は4ニブルグループの先頭にF
・右側は4ニブルグループの末尾にF
・全空白行の次では左側の配置状態が変わる
・状態は画像ごとにリセットする

4. 共通画像は発見の入口

TetrisとFlash Pointには、同一と考えられるコイン画像2個、合計576バイトが含まれています。Flash Point側のFを0へ変換すると、Tetris側の該当範囲と576バイトすべてが一致します。

```
Flash Pointの共通画像
↓ Fを0へ変換
Tetrisの共通画像と576バイト完全一致
```

この一致から、PS2用変換でFが0へ潰されていることと、元のF配置の特徴を推定できました。

ただし、576バイトの画像をテンプレートとしてTetris全体へコピーしたわけではありません。そこで得た短い規則をFlash Pointの69画像・1,967行へ広げて検証しています。

5. Tetrisへの適用

Tetrisでは、tetrisd.p94から58画像の開始位置・幅・高さを読み取り、tetris.objを画像と行に分割します。その後、Flash Pointで確認した規則を各行へ適用します。

```
tetrisd.p94から58画像の配置を取得
↓
tetris.objを画像・行単位に分割
↓
各行を4ニブル単位で解析
↓
Flash Pointで確認した規則に従いFを補完
↓
MAME既知ROMのCRC32・SHA-1と完全一致
```

6. なぜ数万ニブルでも復元できるのか

25,969個のTetris画像ニブルを、25,969回個別に推測しているわけではありません。2,680行に同じ短い規則を適用した結果として、一括でFが補完されます。

> **さらに、同じ規則はFlash Point側で11,819ニブルを補完し、69画像・1,967行すべてを元データへ戻しました。この別作品での一致が、単なるTetris専用の差分合わせではないことを強く裏付けます。**

7. 残る限界

全空白行では、複数のF配置が同じ表示結果になることがあります。そのため、Tetris単体からすべてのF位置を数学的に一意決定できるわけではありません。

正規ROMと全バイト一致する候補を選ぶには、正常なflashp.objに見られる配置規則と、最終的なMAMEのCRC32・SHA-1照合も利用しています。Flash Pointがなければ、見た目上動作するROMは作れても、正規ROMと同一のバイト列まで根拠を持って決めるのは難しかったと考えられます。

まとめ

> **正確な理解は、共通画像の一致から規則を発見し、4ニブル単位の画像構造と配置表を使い、Flash Point全体で規則を検証したうえでTetrisへ適用した、というものです。**

参考

MAME 0.289 System 16B driver
https://github.com/mamedev/mame/blob/mame0289/src/mame/sega/segas16b.cpp
MAME 0.289 Sega 16-bit sprite implementation
https://github.com/mamedev/mame/blob/mame0289/src/mame/sega/sega16sp.cpp